Reviewer Pack — Minimal Number of Integral Points in Quadratic Forms and SAT…

Entry 6964f66ce6c6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 17:49:55 UTC

Minimal Number of Integral Points in Quadratic Forms and SAT Clause Entropy

Entry ID: `6964f66ce6c6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 17:49:55 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Quadratic Forms) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Entropy)

Statement:

> For every Conjunctive Normal Form (CNF) φ with n clauses, the minimal number of integral points in its corresponding quadratic form is linearly correlated with its clause entropy, such that MinIntegralPoints(φ) = Θ(SATEntropy(φ)).

Rationale (proposer's reasoning):

> Quadratic forms provide a rich structure to study integer properties, and their integral points can reveal intricate patterns. SAT clause entropy measures the complexity of the CNF, and this conjecture suggests an intimate connection between these two distinct mathematical domains that could shed light on the inherent complexity of satisfiability problems.

Taxonomy category: `QuadraticForms-SATEntropy` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `271f9fb62db5035b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF φ with n clauses, if the correlation coefficient between MinIntegralPoints(φ) and SATEntropy(φ) is ≥ 0.8 AND the p-value ≤ 0.05 for at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Number of Integral Points AND Quadratic Forms IN title AND SAT Clause Entropy IN title - Quadratic Forms AND Conjunctive Normal Form AND clause entropy - SAT Clause Entropy AND integral points AND quadratic form

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
            cnf.append(clause)
        return cnf

    def quadratic_form(cnf):
        n = len(cnf[0])
        qform = [[0] * n for _ in range(n)]
        for clause in cnf:
            for x in clause:
                for y in clause:
                    if abs(x) == abs(y):
                        qform[abs(x)-1][abs(y)-1] += 1
        return qform

    def min_integral_points(qform):
        n = len(qform)
        det = determinant(qform)
        if det == 0:
            return float('inf')
        return math.ceil(abs(det) ** (1/n))

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
```

```
            return matrix[0][0]
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1) ** j * matrix[0][j] * determinant(submatrix)
        return det

    def sat_entropy(cnf):
        n = len(cnf)
        total_clauses = sum(1 for clause in cnf if any(x != 0 for x in clause))
        entropy = -total_clauses / (n * math.log2(n)) if n > 0 else float('inf')
        return entropy

    n_max = 40
    instances_tested = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in range(5, n_max + 1):
        cnf = generate_cnf(n)
        qform = quadratic_form(cnf)
        min_points = min_integral_points(qform)
        entropy = sat_entropy(cnf)

        if min_points == float('inf'):
            continue

        instances_tested += 1
        total_metric_value += min_points * entropy

    mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0.0
    support_fraction = instances_tested / (n_max - 4)

    if support_fraction < 0.8:
        conjecture_holds = False
        counterexample = "support_fraction_too_low"

    return {
        "metric_name": "MinIntegralPoints * SATEntropy",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)
```

```
    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fractio
    elif any(r["counterexample"] == "support_fraction_too_low" for r in results):
        print("RESULT: FALSIFIED counterexample=\"support_fraction_too_low\" first_failing_seed=<s
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction_too_low")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba5e864b.py", line 101, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba5e864b.py", line 69, in run_trial
    qform = quadratic_form(cnf)
            ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba5e864b.py", line 35, in quadratic_for
    qform[abs(x)-1][abs(y)-1] += 1
    ~~~~~^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide the required correlation coefficients and p-values. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14554
2	propose	ollama_remote	glm4:latest	0	0	13429
3	preregistration	ollama_remote	glm4:latest	0	0	9316

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8461
5	novelty	ollama_remote	glm4:latest	0	0	8855
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15236
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12255
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16407
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10385
10	judge	ollama_remote	glm4:latest	0	0	11678

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 120575 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/6964f66ce6c6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6964f66ce6c6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6964f66ce6c6.tar.gz` (if generated)